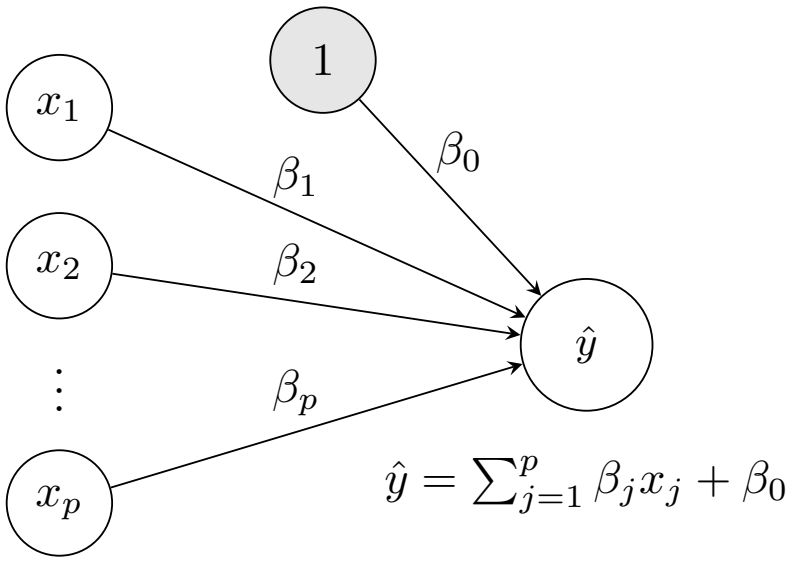
Ukesoppgaver - Uke 5

Lineær regresjon - IN1160

I disse ukesoppgavene skal vi se nærmere på hvordan utregninger med lineær regresjon kan gjøres. Utregning av eksempler for lineær regresjon er en av de beste måtene på å ordentlig forstå hvordan modellen fungerer. Vi har prøvd å velge eksemplene så det ikke skal bli mye eller avansert regning. I oblig 2a vil dere implementere lineær regresjon selv og anvende det på ekte datasett, som godt vil komplementere læringen fra disse ukesoppgavene.

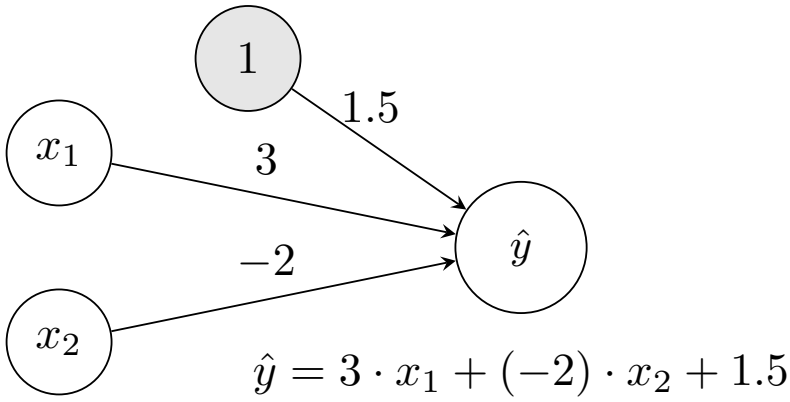
Vi begynner med noen eksempler for å regne ut prediksjonen til en lineær regresjonsmodell. Vi kan generelt sett representere lineær regresjon med diagrammet i Figur 1.



Figur 1: Illustrasjon av en lineær regresjonsmodell.

Her er altså $(x_1, x_2, ...x_p)$ inputen, med p antall forskjellig trekk (features). Alle x-ene i Figur 1 tilhører altså ett datapunkt med p trekk. Koeffisientene er representert med $(\beta_0, \beta_1, ..., \beta_p)$, der β_0 er konstantleddet (biasen). $\hat{y}$ er modellens prediksjon.

La oss nå se på et eksempel. Vi ser på en modell som tar to input, x_1 og x_2. Koeffisientene til disse inputene er henholdsvis 3 og -2 (β_1 og β_2), og konstantleddet er 1.5 (β_0). Dette kan visualiseres som i Figur 2.



Figur 2: Eksempel på en spesifikk lineær regresjonsmodell.

Merk: Oppgavene under bygger på hverandre. Det vil si at dersom man regner feil i en oppgave vil det kunne føre til feil videre i de neste oppgavene. Det er derimot ikke så farlig, siden oppgavene er laget for å gi dere forståelse, ikke for å bli vurdert. Legg allikevel merke til at alle tallene er valgt for å gjøre utregningen og resultatet så rundt som mulig (bortsett fra den aller siste), så dersom dere får mye regning og mange desimaler i resultatet, kan det være lurt å dobbeltsjekke svaret før dere går videre.

Oppgave 5.1 - Prediksjon

Vi bruker modellen i Figur 2. Gitt følgende input for (x_1, x_2), hva blir prediksjonen $\hat{y}$?

a) $\mathbf{x}_1 = (x_{1,1}, x_{1,2}) = (1, 2)$

b) $\mathbf{x}_2 = (x_{2,1}, x_{2,2}) = (1.5, -1)$

c) $\mathbf{x}_3 = (x_{3,1}, x_{3,2}) = (0, 10)$

Oppgave 5.2 - Tapsfunksjon

Vi kan nå bruke prediksjonen til å regne ut *gjennomsnittlig kvadratfeil*, på engelsk kalt *mean squared error* (MSE). Husk at MSE er definert ved:

$$\text{MSE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Her er $\mathbf{y}$ og $\hat{\mathbf{y}}$ lister av henholdsvis y- og $\hat{y}$-verdier, med sammenfallende indekser.

For å regne ut MSE tar vi altså kvadratet av forskjellen mellom hver sanne verdi og dens tilhørende prediksjon, før vi summerer alle disse sammen og deler resultatet på antall par av y og $\hat{y}$. Kvadreringen sørger for at forskjellen alltid er et positivt tall, slik at negative og positive verdier ikke kansellerer hverandre ut.

Gitt de sanne y-verdiene $\mathbf{y} = (y_1, y_2, y_3) = (-0.5, 10, -20.5)$ for de predikerte $\hat{y}$-verdiene i henholdsvis a), b) og c) i oppgave 5.1, hva blir MSE-en?

Oppgave 5.3 - Koeffisientoppdatering

Vi kan nå bruke prediksjonene og de sanne verdiene til å oppdatere koeffisienter med *gradientnedstigning* (*gradient descent* på engelsk). Målet vårt er altså å finne koeffisienter som minimerer tapsfunksjonen (loss-funksjonen) vår, som er MSE. Dette kan vi gjøre med derivasjon. Den deriverte til en funksjon forteller hvor raskt den vokser. Dersom vi deriverer MSE-en med hensyn på en av koeffisientene våre β_j, notert $\frac{\partial \text{MSE}(\mathbf{y},\hat{\mathbf{y}})}{\partial \beta_j}$, får vi et tall som forteller oss *hvor mye MSE-en vil vokse* dersom vi gjør en *liten endring i* β_j. Dette kan regnes ut med vanlige derivasjonsregler, men selve utregningen er ikke pensum i dette faget.

Dette vil vi gjøre for alle koeffisientene våre. Vi bruker symbolet ∂ for å symbolisere at vi deriverer med hensyn på én variabel, og dette heter den *partiellderiverte*. Dersom vi putter alle de partiellderiverte sammen til en vektor får vi *gradienten*. For vårt eksempel kan vi notere dette med:

$$\nabla_\beta \text{MSE}(\mathbf{y}, \hat{\mathbf{y}}) = \left(\frac{\partial \text{MSE}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \beta_0}, \frac{\partial \text{MSE}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \beta_1}, \frac{\partial \text{MSE}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \beta_2}\right)$$

.

Merk at de sanne verdiene $\mathbf{y}$ og prediksjonene $\hat{\mathbf{y}}$ blir håndtert som konstanter i dette uttrykket, siden vi kan regne dem ut som spesifikke tall eller vektorer av tall. Det vi deriverer med hensyn på er altså koeffisientene β.

Det gjenstår å bruke gradient descent til å oppdatere koeffisientene i retningen der den minsker raskest mulig. Gradienten står i retningen der funksjonen *vokser* raskest mulig, så vi vil endre koeffisientene i retning av *den negative gradienten*. Selv om gradienten sier hvilken retning vi må endre alle vektene for å få tapsfunksjonen til å minske, vil disse verdiene endre seg etter vi har oppdatert koeffisientene. Det

er derfor viktig å ikke ta et for stort steg, siden dette kan føre til at vi "hopper over" minimumet av tapsfunksjonen vi prøver å finne. Gradient descent sier at vi skal oppdatere koeffisientene *et lite steg i retning mot den negative gradienten.* Dette gir en iterativ prosess der vi først regner ut hva modellen vår predikerer, deretter finner gradienten og oppdatere vektene, for så å regne ut prediksjonen på nytt med de nye vektene, og så videre. For koeffisient β_j i steg nummer k kan vi uttrykke dette som:

$$\beta_j^{(k)} \leftarrow \beta_j^{(k-1)} - \eta \frac{\partial \text{MSE}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \beta_j^{(k-1)}}$$

Der $\beta_j^{(k)}$ og $\beta_j^{(k-1)}$ er koeffisienten β_j i steg nummer k og $k-1$ (som *ikke* er eksponenter) og η (den greske bokstaven "eta") er *læringsraten*, som kontrollerer hvor stort steg vi tar.

Dersom man bruker derivasjonsregler og fyller inn i oppdateringsregelen for lineær regresjon (selve utregningen er ikke del av pensum) får vi følgende uttrykk (det kommer også en faktor 2, men denne inkluderer vi i læringsraten η):

$$\beta_0^{(k)} \leftarrow \beta_0^{(k-1)} - \eta \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

$$\beta_1^{(k)} \leftarrow \beta_1^{(k-1)} - \eta \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i) x_{i,1}$$

$$\beta_2^{(k)} \leftarrow \beta_2^{(k-1)} - \eta \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i) x_{i,2}$$

Merk at reglene for vektene er like bortsett fra hvilket trekk som brukes i inputen, og regelen for konstantleddet er lik bortsett fra at faktoren til inputen mangler. I vårt eksempel er n, antall datapunkter, lik 3. La η være 0.03 og bruk prediksjonene $\hat{y}_1$, $\hat{y}_2$ og $\hat{y}_3$ fra oppgave 5.1

a) Regn ut den nye verdien til β_0.

b) Regn ut den nye verdien til β_1.

c) Regn ut den nye verdien til β_2.

Oppgave 5.4 - Nye prediksjoner

Med de nye vektene kan vi regne ut prediksjonene på nytt og se om vi får bedre resultat. Vi kan så bruke disse nye resultatene til å regne ut nye vekter en gang til, og så videre, men vi gir oss etter en runde til med prediksjoner og overlater videre beregninger til datamaskiner i oblig 2a.

a) Regn ut prediksjonene for datapunktene $(1, 2)$, $(1.5, -1)$ og $(0, 10)$ fra oppgave 5.1 med de nye vektene fra oppgave 5.3.

b) Regn ut MSE-en (gitt de samme sanne verdiene $\mathbf{y} = (-0.5, 10, -20.5)$ fra oppgave 5.2) med de nye prediksjonene. Hvordan ble resultatet sammenliknet med MSE-en med de forrige vektene? Hvorfor tror du det ble slik?

[Bonus] Oppgave 5.5 - Alternativ læringsrate

Gjenta utregningen i oppgave 5.3 og 5.4 for $\eta = 0.3$ istedenfor $\eta = 0.03$. Hva skjer nå med MSE-en? Hvorfor?